

Testing in Python 3

Unit Testing, TDD, Test Doubles & CI

A Complete Practitioner's Guide for Production Python Engineering

Part of the Series:

End-to-End GenAI & Data Science: Production-Ready Engineering

- ◆ Unit Testing Fundamentals — Vocabulary & Concepts
- ◆ unittest Framework — Fixtures, Assertions & Test Organisation
 - ◆ pytest Framework — Fixtures, Parameterisation & Plugins
- ◆ Test Driven Development (TDD) — Red-Green-Refactor Cycle
 - ◆ Continuous Integration (CI) — Automated Testing Pipelines
- ◆ Test Doubles — Stubs, Fakes, Dummies, Spies & Mocks
 - ◆ Code Coverage, Branch Coverage & Mutation Testing
- ◆ Hard-to-Test Code — Peel, Slice & Monkey Patching Strategies
 - ◆ Approval Testing & Self-Initialising Fakes

SECTION 1 — THEORY

Chapter 1: The Case for Automated Testing

1.1 Why Automated Testing Is Non-Negotiable

In professional software engineering — and particularly in data science and machine learning systems — the cost of an undetected bug grows exponentially with the distance between its introduction and its discovery. A bug caught by a developer's own test suite costs minutes to fix. The same bug discovered in a deployed model serving production traffic can cost hours of investigation, corrupt downstream datasets, invalidate training pipelines, and erode stakeholder trust.

Automated tests are executable specifications: they codify precisely what a piece of code is supposed to do and verify that claim every time they run. Unlike documentation (which becomes stale) or manual testing (which is slow, inconsistent, and expensive to scale), automated tests are cheap to execute, deterministic, and become more valuable as a codebase grows.

Without Automated Tests	With Automated Tests
Changes are made cautiously, for fear of breaking something	Changes are made confidently — the test suite immediately reports breakages
Bugs are discovered by users in production	Bugs are caught by tests before code is merged
Refactoring is risky and avoided	Refactoring is safe — tests verify behaviour is preserved
Collaboration is slow — reviewers cannot verify correctness	Collaboration is fast — CI runs tests on every pull request
Debugging is expensive — cause is often distant from symptom	Debugging is cheap — failing test directly pinpoints the broken unit

1.2 What Is Unit Testing?

A unit test is an automated test that verifies the behaviour of a single, isolated unit of code — typically a function, method, class, or small module. The defining characteristic of a true unit test is isolation: the unit under test is exercised independently of its collaborators (databases, file systems, network services, external APIs). Where collaborators are required, they are replaced with controlled substitutes called test doubles (explored in depth in Chapter 7).

This isolation principle has two important consequences. First, unit tests are fast — running hundreds or thousands of them takes seconds because they perform no I/O. Second, when a unit test fails, the failing test's name and location precisely identify the broken component, dramatically accelerating debugging.

```
# The simplest possible unit test: no framework required
def add(a: int, b: int) -> int:
    """Return the sum of two integers."""
    return a + b

# assert statement: raises AssertionError if condition is False
assert add(2, 3) == 5          # passes silently
assert add(-1, 1) == 0        # passes silently
assert add(0, 0) == 0         # passes silently

# This would fail, exposing the bug immediately:
# assert add(2, 2) == 5        # AssertionError (2+2 == 4, not 5)
```

Scope Definition: Unit testing tests individual functions/methods in isolation. Integration testing tests multiple components working together (e.g., a service calling a real database). End-to-end testing tests the full system from the user's perspective. This chapter focuses primarily on unit testing, which forms the foundation of the testing pyramid.

1.3 The Economics of Testing

A common objection to writing tests is that they 'slow down development'. This objection confuses short-term velocity with long-term productivity. Research across the software engineering literature consistently shows that projects with comprehensive test suites experience:

- **Lower defect density:** fewer bugs reaching production per thousand lines of code.
- **Faster long-term delivery:** reduced time spent debugging and firefighting as the codebase scales.
- **Lower onboarding cost:** new team members can make changes confidently once they understand what the tests verify.
- **Safer refactoring:** internal restructuring of code without changing external behaviour is validated automatically.

For data science and GenAI projects specifically, the stakes are elevated: an untested data preprocessing step can silently introduce bias into a model's training data; an untested model serialisation routine can produce corrupted model artefacts; an untested API endpoint can serve incorrect predictions at scale. Tests are not a luxury in production ML systems — they are a prerequisite.

Chapter 2: Testing Vocabulary & Concepts

2.1 Core Terminology

A shared, precise vocabulary enables effective communication within engineering teams about testing strategy and test quality. The following terms constitute the essential testing lexicon:

2.1.1 Unit Under Test (UUT)

The Unit Under Test is the specific piece of code being exercised by a given test. It is the subject of the test — the function, method, class, or module whose behaviour is being verified. Identifying the UUT clearly before writing a test is the first step in good test design: a test that exercises multiple units simultaneously is harder to name, harder to maintain, and harder to debug when it fails.

2.1.2 Test Case

A test case is a single, discrete test scenario: a specific combination of inputs, an execution of the UUT, and an assertion about the expected output or state change. A well-designed test case:

- Tests exactly one behaviour or scenario.
- Has a meaningful name that describes what is being tested and under what conditions.
- Is deterministic — it produces the same result every time it runs.
- Is independent — it does not depend on the execution order of other tests.

```
# Good test case: one behaviour, meaningful name, clear structure
def test_phonebook_lookup_returns_correct_number_for_known_name():
    phonebook = Phonebook()
    phonebook.add('Bob', '12345')    # Arrange
    number = phonebook.lookup('Bob') # Act
    assert number == '12345'         # Assert
```

2.1.3 Test Runner

A test runner is a tool that discovers, executes, and reports on a collection of test cases. Modern test runners provide:

- **Automatic test discovery:** finding test files and functions based on naming conventions.
- **Parallel execution:** running tests concurrently to reduce total suite runtime.
- **Structured output:** reporting pass/fail counts, timing, error messages, and stack traces.
- **Exit codes:** returning non-zero on failure, enabling CI pipeline integration.

Python's two primary test runners are unittest (standard library) and pytest (third-party, but near-universal in professional Python development).

2.1.4 Test Suite

A test suite is an organised collection of test cases that are executed together. Suites provide structure for large codebases: individual functions have their own test cases; a module has a test suite; an entire project has a master test suite. Test runners discover and execute suites automatically when given a root directory.

2.1.5 Test Fixture

A test fixture is the setup and teardown logic required to put the system into a known, clean state before each test runs, and to restore it to a clean state afterward. Without fixtures, tests that share resources (database connections, temporary files, in-memory data structures) can produce order-dependent results — passing when run in one order and failing in another.

Fixtures enforce test isolation: each test begins from an identical, predictable starting state.

```
import unittest

class PhonebookTest(unittest.TestCase):

    def setUp(self): # runs BEFORE each test method
        """Create a fresh Phonebook for each test — guaranteed
isolation."""
        self.phonebook = Phonebook()

    def tearDown(self): # runs AFTER each test method, even on failure
        """Clean up any resources allocated during the test."""
        self.phonebook.clear() # release resources

    def test_lookup_existing_name(self):
        self.phonebook.add('Bob', '12345')
        self.assertEqual(self.phonebook.lookup('Bob'), '12345')

    def test_lookup_missing_name_raises(self):
        with self.assertRaises(KeyError):
            self.phonebook.lookup('Alice') # Alice was never added
```

2.2 The Arrange-Act-Assert (AAA) Pattern

The Arrange-Act-Assert pattern is the universally accepted structural template for writing readable, well-organised test cases. It imposes a clear three-phase structure that makes the purpose of every test immediately apparent to any reader:

Phase	Responsibility	What Goes Here
Arrange	Set up the preconditions for the test	Create objects, configure mocks, load test data, set environment variables
Act	Execute the unit under test	Call the function, invoke the method, trigger the event being tested
Assert	Verify the outcome	Check return values, verify state changes, confirm exceptions were raised

```
def test_model_prediction_returns_float_between_zero_and_one():
```

```
# — Arrange —————
model = load_trained_classifier('model.pkl')
sample_features = [5.1, 3.5, 1.4, 0.2] # a single Iris measurement

# — Act —————
probability = model.predict_proba([sample_features])[0][1]

# — Assert —————
assert isinstance(probability, float)
assert 0.0 <= probability <= 1.0
```

Best Practice: If the Assert phase requires more than two or three assertions, the test is likely verifying too many behaviours simultaneously. Split it into multiple focused test cases. A good test is a single falsifiable claim about the unit's behaviour.

Chapter 3: The unittest Framework

3.1 Overview & Design

The unittest module is Python's built-in testing framework, included in the standard library since Python 2.1. Its design is directly inspired by JUnit (Java's testing framework), which in turn draws from Smalltalk's SUnit. As a result, unittest uses an object-oriented, class-based architecture: each group of related test cases is organised as a subclass of `unittest.TestCase`.

While pytest has largely superseded unittest in modern Python development, unittest remains important for several reasons: it requires no external dependencies, it is the framework used in many legacy codebases, it underpins Python's doctest module, and understanding its concepts (setUp, tearDown, assert methods) is foundational to understanding all testing frameworks.

3.2 Writing unittest Tests

```
import unittest

class Phonebook:
    """A simple in-memory name-to-number mapping."""

    def __init__(self):
        self._entries: dict[str, str] = {}

    def add(self, name: str, number: str) -> None:
        self._entries[name] = number

    def lookup(self, name: str) -> str:
        if name not in self._entries:
            raise KeyError(f'No entry for {name!r}')
        return self._entries[name]
```

```
def names(self) -> list[str]:
    return list(self._entries.keys())

def is_consistent(self) -> bool:
    """Return True if no entry is a prefix of another."""
    names = self.names()
    for i, name_a in enumerate(names):
        for name_b in names[i+1:]:
            if name_a.startswith(name_b) or
name_b.startswith(name_a):
                return False
    return True

class PhonebookTest(unittest.TestCase):
    """Test suite for the Phonebook class."""

    def setUp(self):
        """Create a fresh Phonebook before each test."""
        self.phonebook = Phonebook()

    def tearDown(self):
        """Explicit cleanup (not strictly needed for pure in-memory
objects)."""
        pass

    # — Basic lookup —————
    def test_lookup_by_name_returns_correct_number(self):
        self.phonebook.add('Bob', '12345')
        number = self.phonebook.lookup('Bob')
        self.assertEqual(number, '12345')

    def test_lookup_missing_name_raises_key_error(self):
        with self.assertRaises(KeyError):
            self.phonebook.lookup('Alice')

    # — Membership —————
    def test_names_contains_added_entry(self):
        self.phonebook.add('Bob', '12345')
        self.assertIn('Bob', self.phonebook.names())

    # — Consistency —————
    def test_empty_phonebook_is_consistent(self):
        self.assertTrue(self.phonebook.is_consistent())

    def test_phonebook_with_distinct_names_is_consistent(self):
        self.phonebook.add('Bob', '12345')
        self.phonebook.add('Alice', '99999')
        self.assertTrue(self.phonebook.is_consistent())

    def test_phonebook_with_prefix_names_is_not_consistent(self):
        self.phonebook.add('Bob', '12345')
        self.phonebook.add('Bob Smith', '99999') # 'Bob' is prefix of
'Bob Smith'
```

```
self.assertFalse(self.phonebook.is_consistent())

if __name__ == '__main__':
    unittest.main()
```

3.3 The unittest Assertion Library

`unittest.TestCase` provides a rich library of assertion methods that produce clear, informative failure messages when they fail — far more useful than a bare `assert` statement, which reports only `AssertionError` without context:

Assertion Method	Verifies	Failure Message Includes
<code>assertEqual(a, b)</code>	<code>a == b</code>	Both actual and expected values
<code>assertNotEqual(a, b)</code>	<code>a != b</code>	The unexpectedly equal values
<code>assertTrue(x)</code>	<code>bool(x)</code> is True	The actual falsy value
<code>assertFalse(x)</code>	<code>bool(x)</code> is False	The actual truthy value
<code>assertRaises(Exc, fn, ...)</code>	<code>fn()</code> raises <code>Exc</code>	The exception type and actual exception
<code>assertIn(member, container)</code>	member in container	The member and container
<code>assertNotIn(member, container)</code>	member not in container	The member and container
<code>assertIsNone(x)</code>	<code>x</code> is None	The actual non-None value
<code>assertIsNotNone(x)</code>	<code>x</code> is not None	Indicates None was received
<code>assertAlmostEqual(a, b, places)</code>	$\text{abs}(a-b) < 10^{-\text{places}}$	Both values and their difference
<code>assertGreater(a, b)</code>	<code>a > b</code>	Both values
<code>assertIsInstance(obj, cls)</code>	<code>isinstance(obj, cls)</code>	The actual type and expected class

Float Comparisons: Never use `assertEqual()` to compare floating-point numbers — floating-point arithmetic introduces tiny rounding errors that make exact equality comparisons unreliable. Always use `assertAlmostEqual(a, b, places=7)` or, for NumPy arrays in ML projects, `numpy.testing.assert_allclose(actual, expected, rtol=1e-5)`.

Chapter 4: The pytest Framework

4.1 Why pytest?

pytest is the dominant testing framework in modern Python development, and for good reason. It eliminates the boilerplate class structure required by unittest, leverages Python's native assert statement (with automatic introspection to produce informative failure messages), and provides a fixture system that is significantly more flexible and composable than unittest's setUp/tearDown model.

Feature	unittest	pytest
Setup required	import unittest; class inherits TestCase	pip install pytest
Test organisation	Class-based (must subclass TestCase)	Function-based or class-based (no inheritance)
Assertions	self.assertEqual(), self.assertTrue(), etc.	Built-in Python assert statement
Fixtures	setUp() / tearDown() (class-level)	@pytest.fixture (function-level, composable)
Parameterisation	Manual (requires subclassing or data loop)	@pytest.mark.parametrize (native, declarative)
Test discovery	Explicit or naming convention	Automatic: test_*.py files, test_* functions
Failure messages	Generic AssertionError	Rich diff output with actual vs expected
Plugin ecosystem	Limited	500+ plugins (coverage, async, Django, etc.)

4.2 Writing pytest Tests

The most immediate difference when switching to pytest is the elimination of class boilerplate. Test functions are plain Python functions starting with test_; assertions use Python's native assert statement:

```
# test_phonebook.py - plain functions, no class required

from phonebook import Phonebook

def test_lookup_by_name_returns_correct_number():
    phonebook = Phonebook()
    phonebook.add('Bob', '12345')
    assert phonebook.lookup('Bob') == '12345'

def test_lookup_missing_name_raises_key_error():
    phonebook = Phonebook()
    with pytest.raises(KeyError) as exc_info:
        phonebook.lookup('Alice')
    assert 'Alice' in str(exc_info.value) # verify error message content
```

```
def test_names_contains_added_entry():
    phonebook = Phonebook()
    phonebook.add('Bob', '12345')
    assert 'Bob' in phonebook.names()
```

When a pytest assertion fails, it produces a rich diff rather than a bare AssertionError:

```
# Example of pytest's rich assertion failure output:
# FAILED test_phonebook.py::test_lookup_by_name_returns_correct_number
# AssertionError: assert '99999' == '12345'
#   Left:  '99999'
#   Right: '12345'
```

4.3 pytest Fixtures — Composable Test Setup

pytest's fixture system is its most powerful feature. A fixture is a function decorated with `@pytest.fixture` that returns (or yields) a value representing a piece of test infrastructure. When a test function declares a parameter whose name matches a fixture, pytest automatically calls the fixture function and injects the result — a mechanism called dependency injection.

```
import pytest
from phonebook import Phonebook

# — Basic fixture —————
@pytest.fixture
def phonebook():
    """Provide a fresh Phonebook for each test that requests it."""
    return Phonebook()

# — Fixture with cleanup using yield —————
@pytest.fixture
def phonebook_with_cleanup(tmp_path):
    """
    Phonebook backed by a temporary file.
    tmp_path is a built-in pytest fixture that provides a unique temp
    directory.
    Everything after yield runs as teardown — even if the test fails.
    """
    pb = Phonebook(storage_path=tmp_path / 'phonebook.json')
    yield pb
    pb.close()    # teardown: runs unconditionally after the test

# — Fixture composition: fixtures can depend on other fixtures —————
@pytest.fixture
def phonebook_with_bob(phonebook):
    """Extend the phonebook fixture with a pre-populated entry."""
    phonebook.add('Bob', '12345')
    return phonebook
```

```
# — Tests use fixtures by parameter name —————
def test_lookup_existing_name(phonebook):
    phonebook.add('Bob', '12345')
    assert phonebook.lookup('Bob') == '12345'

def test_lookup_in_prepopulated_book(phonebook_with_bob):
    # No setup needed – phonebook_with_bob already has Bob
    assert phonebook_with_bob.lookup('Bob') == '12345'
```

Fixture Scope	Created Once Per...	Use Case
function (default)	Test function call	Default: isolates each test fully
class	Test class	Shared setup for a cohesive group of related tests
module	Module (.py file)	Expensive shared resources: database connections
session	pytest session (entire test run)	Very expensive one-time setup: ML model loading

4.4 Exception Testing in pytest

`pytest.raises()` is the pytest equivalent of unittest's `assertRaises()`. It is used as a context manager that passes only if the specified exception is raised within its block:

```
import pytest

def test_lookup_missing_name_raises_key_error(phonebook):
    """Verify that looking up an absent name raises KeyError."""
    with pytest.raises(KeyError):
        phonebook.lookup('NonExistent')

def test_exception_message_contains_name(phonebook):
    """Verify the error message includes the searched name."""
    with pytest.raises(KeyError) as exc_info:
        phonebook.lookup('Ghost')
    # exc_info.value is the actual exception instance
    assert 'Ghost' in str(exc_info.value)

def test_invalid_input_raises_type_error():
    """Verify that non-string input raises TypeError."""
    with pytest.raises(TypeError, match=r'Expected str, got int'):
        phonebook.add(name=123, number='555-0100') # type violation
```

4.5 Parameterised Testing

Parameterised tests execute the same test logic against multiple input/output combinations, eliminating duplicated test code while increasing coverage. This is particularly valuable in data science contexts where a function must behave correctly across a range of valid inputs and edge cases:

```
import pytest

# — @pytest.mark.parametrize: run one test with multiple datasets —
@pytest.mark.parametrize(
    'a, b, expected',
    [
        (1, 2, 3),      # standard positive integers
        (2, 3, 5),      # another standard case
        (5, 5, 10),     # equal operands
        (-1, 1, 0),     # negative + positive
        (0, 0, 0),      # both zero
        (100, -50, 50), # large values, negative result non-zero
    ]
)

def test_add_produces_correct_sum(a, b, expected):
    assert add(a, b) == expected

# — Parametrize for edge cases in ML preprocessing —————
@pytest.mark.parametrize(
    'input_score, expected_status',
    [
        (100.0, 'PASS'), # perfect score
        (70.0, 'PASS'),  # exactly at threshold
        (69.9, 'FAIL'),  # just below threshold
        (0.0, 'FAIL'),   # minimum possible
        (50.0, 'FAIL'),  # midpoint
    ]
)

def test_grade_classification(input_score, expected_status):
    result = classify_score(input_score, threshold=70.0)
    assert result == expected_status
```

Data Science Application: Parameterised tests are the natural mechanism for testing data preprocessing functions against multiple data types, missing value scenarios, and edge cases (empty arrays, single elements, very large values). They are also the right tool for testing model evaluation functions across multiple metric thresholds.

4.6 Skipping Tests

pytest provides decorators to conditionally skip tests, enabling developers to mark work-in-progress tests, skip tests on incompatible platforms, or skip tests that require unavailable external resources:

```
import pytest
import sys

# — Unconditional skip: marks test as WIP without failing —————
@pytest.mark.skip(reason='Feature not yet implemented')
def test_new_llm_feature():
    pass # placeholder – test will be implemented with the feature

# — Conditional skip: skip if runtime condition is not met —————
@pytest.mark.skipif(
    sys.version_info < (3, 10),
    reason='Match statement requires Python 3.10+'
)
def test_pattern_matching_feature():
    ...

# — Skip if optional dependency is unavailable —————
try:
    import torch
    TORCH_AVAILABLE = True
except ImportError:
    TORCH_AVAILABLE = False

@pytest.mark.skipif(not TORCH_AVAILABLE, reason='PyTorch not installed')
def test_neural_network_forward_pass():
    ...

# — xfail: mark a test expected to fail (known bug, pending fix) —
@pytest.mark.xfail(reason='Upstream bug in tokenizer, ticket #1234')
def test_tokeniser_edge_case():
    assert tokenize('') == [] # known to fail until bug is fixed
```

4.7 Organising Tests — Project Structure

A well-organised test directory is as important as well-organised source code. The recommended project structure separates tests from production code completely, mirrors the package structure for easy navigation, and uses descriptive naming:

```
my_project/
├── src/
│   └── my_project/
│       ├── __init__.py
│       ├── preprocessing.py
│       ├── models.py
│       ├── evaluation.py
│       └── api/
│           ├── __init__.py
│           └── endpoints.py
├── tests/
│   ├── conftest.py           # shared fixtures available to all tests
│   └── unit/
```

```
├── test_preprocessing.py
├── test_models.py
├── test_evaluation.py
├── integration/
│   └── test_pipeline.py
├── e2e/
│   └── test_api.py
├── pyproject.toml
└── pytest.ini                # pytest configuration
```

The `conftest.py` file is pytest's shared fixture module — fixtures defined here are automatically available to all tests in the same directory and its subdirectories, without requiring explicit import. This is the correct location for project-wide fixtures such as database connections, test clients, and loaded ML models.

Chapter 5: Test Driven Development (TDD)

5.1 What Is TDD?

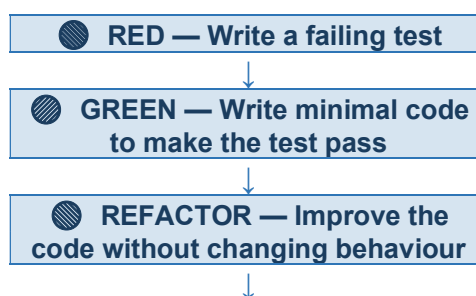
Test Driven Development is a software development methodology in which tests are written before the implementation they test. This inversion of the traditional write-code-then-test sequence has profound effects on code design, correctness, and maintainability. TDD is not primarily a testing technique — it is a design technique that uses tests as its primary feedback mechanism.

"TDD is a way of managing fear during programming." — Kent Beck, creator of TDD

The core insight of TDD is that writing a test before writing any implementation forces the developer to think precisely about the interface they are designing: What should this function be called? What parameters should it accept? What should it return? What exceptions should it raise? These questions, answered in the form of a failing test, produce clearer, more focused implementation code than would emerge from writing implementation first.

5.2 The Red-Green-Refactor Cycle

TDD operates through a rapid, iterative three-phase cycle known as Red-Green-Refactor:



Repeat

- **Red:** Write a test case that describes a specific piece of desired behaviour. Run it. It must fail — if it passes without any implementation, either the test is wrong or the behaviour already exists. A failing test is confirmation that the test is actually testing something.
- **Green:** Write the minimum amount of code necessary to make the failing test pass. Do not write more than needed — do not add features, handle edge cases not covered by tests, or premature optimise. The goal is a passing test, nothing more.
- **Refactor:** With a passing test as a safety net, improve the implementation: remove duplication, clarify names, extract helper functions, reorganise structure. Run the tests after every change to confirm the refactoring has not broken anything.

```
# — TDD example: building a Stack data structure —————

# STEP 1 — RED: Write the test first (Stack class does not exist yet)
def test_new_stack_is_empty():
    stack = Stack()
    assert stack.is_empty() is True

# Run: FAILS with NameError: name 'Stack' is not defined ✓

# STEP 2 — GREEN: Write minimal implementation
class Stack:
    def __init__(self):
        self._items = []

    def is_empty(self) -> bool:
        return len(self._items) == 0

# Run: PASSES ✓

# STEP 3 — RED: Add next test
def test_push_makes_stack_non_empty():
    stack = Stack()
    stack.push(42)
    assert stack.is_empty() is False

# STEP 4 — GREEN: Extend implementation
class Stack: # updated
    def __init__(self):
        self._items = []

    def is_empty(self) -> bool:
        return len(self._items) == 0

    def push(self, item) -> None:
        self._items.append(item)

# STEP 5 — REFACTOR: No duplication yet, continue the cycle
```

```
# Next test: pop() returns last pushed item — write test, fail,  
implement, pass...
```

TDD in Data Science: TDD is directly applicable to data science code. Write tests for preprocessing functions (does `normalise_features()` produce values in `[0,1]`?), evaluation functions (does `compute_f1()` return 1.0 for perfect predictions?), and pipeline steps (does the pipeline handle missing values without raising?). The model training loop itself is harder to TDD, but the surrounding infrastructure is not.

5.3 Benefits of TDD

- **Design clarity:** Tests force you to define the public interface before implementation, naturally producing smaller, more focused functions with cleaner interfaces.
- **Built-in regression suite:** Every test written in TDD becomes a permanent regression test. By the time a feature is complete, it already has comprehensive test coverage.
- **Refactoring safety:** The existing test suite provides a safety net for every refactoring operation, enabling continuous improvement of internal code quality without fear of breaking external behaviour.
- **Documentation:** The test suite doubles as executable documentation — each test case demonstrates exactly how the code is intended to be used.

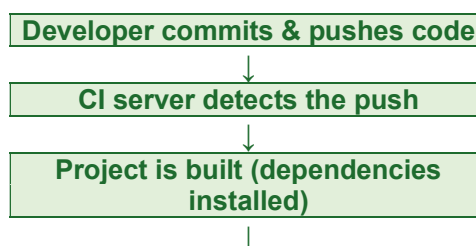
Chapter 6: Continuous Integration (CI)

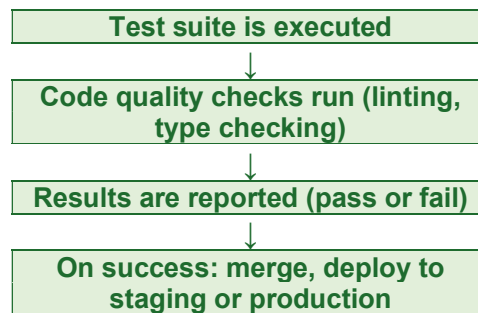
6.1 What Is Continuous Integration?

Continuous Integration (CI) is the software engineering practice of frequently integrating individual developer changes into the shared main branch of a codebase — and automatically verifying each integration through a suite of automated tests, linters, and quality checks. The word 'continuous' is not hyperbole: modern CI systems trigger their automated pipeline on every git push, running the full test suite within minutes.

The foundational principle of CI is that integration problems are easier and cheaper to fix when they are detected immediately after introduction, rather than weeks or months later when multiple conflicting changes have accumulated. CI turns integration from a periodic, painful, multi-day event into a routine, automated, seconds-to-minutes operation.

6.2 The CI Workflow





6.3 CI for Python Projects — GitHub Actions Example

GitHub Actions is the most widely used CI platform for open-source Python projects. Workflows are defined as YAML files in the `.github/workflows/` directory:

```
# .github/workflows/test.yml
name: Python Test Suite

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ['3.10', '3.11', '3.12'] # test on multiple
        versions

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }

      - name: Install dependencies
        run: |
          pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest pytest-cov black flake8 mypy

      - name: Lint with flake8
        run: flake8 src/ tests/

      - name: Type check with mypy
        run: mypy src/
```

```
- name: Run tests with coverage
  run: pytest tests/ --cov=src --cov-report=xml --cov-fail-under=80

- name: Upload coverage report
  uses: codecov/codecov-action@v4
```

CI Gate: The `--cov-fail-under=80` flag fails the CI pipeline if test coverage drops below 80%. This is the standard mechanism for enforcing coverage thresholds as a merge requirement. In production ML projects, coverage targets of 85-90% for business logic and data processing code are common.

6.4 Regression Testing

Regression testing is the practice of re-running the full test suite after any code change, to verify that previously working functionality has not been broken — 'regressed' — by the change. In a CI-based workflow, every commit automatically triggers a regression test run: the test suite exists precisely to catch regressions the moment they are introduced.

Regressions are particularly insidious in ML pipelines because they may not manifest as crashes or exceptions: a data preprocessing bug might silently shift the distribution of features, causing a model that was accurate on the validation set to degrade on production data over time. Regression tests that verify statistical properties of processed data (mean, variance, data type, shape) catch these silent failures.

Chapter 7: Test Doubles

7.1 What Are Test Doubles?

A test double is any object that stands in for a real dependency during a test. The term, coined by Gerard Meszaros in *xUnit Test Patterns*, is a deliberate analogy to a stunt double in film production — an object that performs the same role as the real thing for the purpose of the test, without being the real thing.

Test doubles are used when the real dependency is:

- **Unavailable:** a third-party API, a production database, an external sensor, a hardware device.
- **Slow:** a network service, a large database query, a model training loop.
- **Non-deterministic:** a service that returns different values each call (random number generators, timestamps, weather APIs).
- **Dangerous:** a payment gateway, an email sender, a file deletion service — things you cannot call in a test without real consequences.
- **Not yet implemented:** a collaborator under development by another team.

Test Double Type	Has Logic?	Records Calls?	Verifies Calls?	Causes Test Failure?	Primary Use
Dummy	No	No	No	No	Satisfy parameter requirements
Stub	No (predefined)	No	No	No	Provide controlled return values
Fake	Yes (simplified)	No	No	No	Replace heavy dependencies with lightweight equivalents
Spy	Partial	Yes	No	No	Verify interactions after the fact
Mock	Partial	Yes	Yes	Yes	Specify and verify expected interactions

7.2 Dummy

A dummy is the simplest test double: a placeholder object passed to satisfy a function's parameter signature, but never actually used by the code path being tested. Dummies are common when a function requires a parameter that is irrelevant to the specific behaviour being tested.

```
# The training pipeline requires a logger, but our test only cares
# about the model's output – we don't care about logging behaviour.

dummy_logger = None    # Python's simplest dummy

# Or a more explicit dummy class:
class DummyLogger:
    def log(self, *args, **kwargs):
        pass    # does nothing – just satisfies the interface

def test_model_returns_prediction_for_valid_input():
    model = SentimentClassifier(logger=DummyLogger())
    result = model.predict('Python is great')
    assert result in ('positive', 'negative', 'neutral')
```

7.3 Stub

A stub replaces a dependency with a simpler object that returns predefined, hardcoded values. Unlike a dummy, a stub is actively called by the code under test — it simply returns a controlled

value rather than making a real external call. Stubs are the primary mechanism for isolating a unit from its non-deterministic or slow dependencies.

```
class StubSensor:
    """Stub sensor that always reports a fixed pressure value."""

    def sample_pressure(self) -> float:
        return 17.0 # predefined, deterministic return value

class DiagnosticSystem:
    """System that raises an alarm if pressure exceeds a threshold."""

    def __init__(self, sensor):
        self._sensor = sensor

    def is_pressure_critical(self, threshold: float) -> bool:
        return self._sensor.sample_pressure() > threshold

def test_alarm_triggered_when_pressure_exceeds_threshold():
    # Stub returns 17.0 – no real sensor hardware needed
    system = DiagnosticSystem(sensor=StubSensor())
    assert system.is_pressure_critical(threshold=15.0) is True

def test_no_alarm_when_pressure_below_threshold():
    system = DiagnosticSystem(sensor=StubSensor())
    assert system.is_pressure_critical(threshold=20.0) is False
```

7.4 Fake

A fake is a test double that contains a working (but simplified) implementation of the dependency it replaces. Unlike a stub, a fake has real logic — it just uses a lighter-weight implementation than the real dependency. Fakes are appropriate when the dependency's behaviour is non-trivial and cannot be reduced to a single predefined return value.

```
# — Fake: in-memory database replacing a real PostgreSQL database —
class FakeDatabase:
    """
    In-memory dict-backed database.
    Contains real CRUD logic, but stores data in memory rather than on
    disk.
    Not suitable for production – too slow, no persistence, no
    concurrency.
    """

    def __init__(self):
        self._store: dict = {}

    def save(self, key: str, value: dict) -> None:
        self._store[key] = value
```

```
def find(self, key: str) -> dict | None:
    return self._store.get(key)

def delete(self, key: str) -> None:
    self._store.pop(key, None)

def count(self) -> int:
    return len(self._store)

# — Fake: io.StringIO replacing a real file —————
import io

def test_csv_parser_reads_correct_record_count():
    fake_file = io.StringIO('name,score\nAlice,92\nBob,78\nCarol,85')
    parser = CsvParser(fake_file)    # real CSV logic, fake file object
    records = parser.parse()
    assert len(records) == 3
```

7.5 Spy

A spy is a test double that wraps a real (or fake) object and records information about the calls made to it — which methods were called, how many times, and with what arguments. This recorded information can be inspected after the fact in assertions. Unlike a mock, a spy does not fail immediately when unexpected calls occur; it observes silently and allows assertions to be made afterward.

```
class SpyNotificationService:
    """Spy: records all calls to notify() for later assertion."""

    def __init__(self):
        self.calls: list[dict] = []

    def notify(self, recipient: str, message: str) -> None:
        self.calls.append({'recipient': recipient, 'message': message})
        # Optionally, also call the real implementation if needed

    @property
    def call_count(self) -> int:
        return len(self.calls)

    def was_called_with(self, recipient: str) -> bool:
        return any(c['recipient'] == recipient for c in self.calls)

def test_alert_sends_notification_to_on_call_engineer():
    spy_notifier = SpyNotificationService()
    alert_system = AlertSystem(notifier=spy_notifier)

    alert_system.trigger_critical_alert(message='Model accuracy dropped
below 80%')

    # Assert AFTER the fact using the spy's recorded calls
```

```
assert spy_notifier.call_count == 1
assert spy_notifier.was_called_with('on_call_engineer@company.com')
```

7.6 Mock

A mock is the most sophisticated test double. Like a spy, it records interactions. But additionally, a mock specifies up-front what interactions are expected, and it automatically fails the test if those expectations are not met — including if unexpected calls occur. Python's standard library provides `unittest.mock`, which can generate mocks for any Python object:

```
from unittest.mock import MagicMock, patch, call

# — MagicMock: auto-generated mock for any object —————
def test_alert_system_notifies_correct_recipient():
    mock_notifier = MagicMock()    # auto-generates all methods

    alert_system = AlertSystem(notifier=mock_notifier)
    alert_system.trigger_critical_alert('Accuracy below threshold')

    # Verify the exact call: method name, arguments, and call count
    mock_notifier.notify.assert_called_once_with(
        recipient='on_call@company.com',
        message='Accuracy below threshold'
    )

# — patch: replace a real object with a mock for the test duration —
from unittest.mock import patch

def test_training_pipeline_calls_model_save():
    with patch('mymodule.ModelSerializer.save') as mock_save:
        pipeline = TrainingPipeline()
        pipeline.run(epochs=1)

        mock_save.assert_called_once()    # verify save was called
        args, kwargs = mock_save.call_args
        assert 'model_path' in kwargs    # verify the correct argument

# — return_value: configure what the mock returns when called ———
def test_prediction_pipeline_uses_model_output():
    mock_model = MagicMock()
    mock_model.predict.return_value = [0.92, 0.08]    # simulated output

    pipeline = InferencePipeline(model=mock_model)
    label = pipeline.classify(['feature1', 'feature2'])

    assert label == 'positive'    # verify downstream logic is correct
```

Mock vs Stub: A common confusion: stubs provide inputs to the UUT (controlled return values); mocks verify outputs from the UUT (that the UUT called its dependencies)

correctly). Use stubs when you want to control what the UUT receives. Use mocks when you want to verify what the UUT does to its collaborators.

Chapter 8: Test Coverage, Branch Coverage & Mutation Testing

8.1 Code Coverage

Code coverage measures the percentage of production code that is executed — 'covered' — when the test suite runs. It is typically expressed as a statement coverage percentage: the fraction of executable lines that were reached at least once during testing. Coverage is measured using tools such as coverage.py (integrated with pytest via the pytest-cov plugin).

```
# Install:
# pip install pytest-cov

# Run tests with coverage report:
# pytest --cov=src --cov-report=html --cov-report=term-missing

# The --cov-report=term-missing flag shows which specific lines are NOT
covered:
# Name                               Stmts   Miss  Cover   Missing
# -----
# src/preprocessing.py               45      7    84%    23-25, 67, 89-91

# Fail if coverage drops below 85%:
# pytest --cov=src --cov-fail-under=85
```

While coverage is a useful indicator, it has well-understood limitations. A line of code being executed during a test does not guarantee that its behaviour has been verified — the test may execute a line without making any assertion about its effect. A test suite can achieve 100% coverage while verifying almost nothing. Coverage is a necessary but not sufficient condition for test quality.

8.2 Branch Coverage

Branch coverage extends statement coverage by requiring that every possible branch of a decision point (if/else, try/except, ternary expressions) be exercised. A line with an if statement has two branches: the True path and the False path. Statement coverage considers the line covered if the if is executed at all; branch coverage requires both the True and False paths to be exercised.

```
def classify_score(score: float) -> str:
    if score >= 70:          # branch: True when score >= 70
        return 'PASS'      # branch: False when score < 70
```

```
    return 'FAIL'

# Statement coverage: a single test with score=80 covers both lines
# (the if line and the return 'PASS' line) – 100% statement coverage

# Branch coverage: requires TWO tests:
def test_score_at_threshold():      # exercises the True branch
    assert classify_score(70.0) == 'PASS'

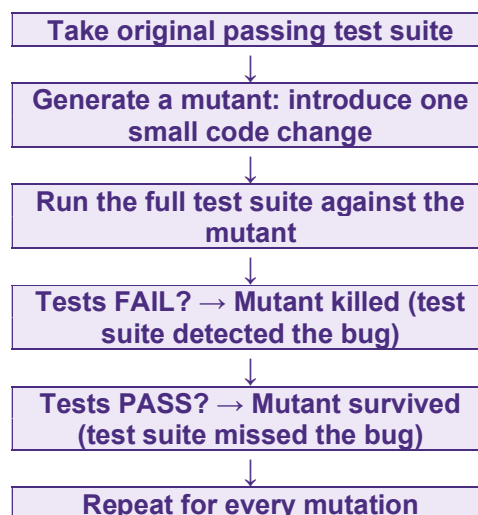
def test_score_below_threshold():   # exercises the False branch
    assert classify_score(69.9) == 'FAIL'

# Enable branch coverage in pytest-cov:
# pytest --cov=src --cov-branch
```

Production Target: For data science and ML projects, target 80-85% statement coverage with branch coverage enabled. Focus coverage effort on business logic, data transformation functions, and error handling paths. Do not target 100% — it incentivises writing tests for trivial getters/setters and `__repr__` methods rather than for meaningful behaviour.

8.3 Mutation Testing

Mutation testing is the most rigorous — and most computationally expensive — technique for measuring test quality. Instead of measuring how much code the tests execute, it measures whether the tests can actually detect faults in the code. The technique works by automatically introducing small, syntactic changes ('mutations') to the production code — flipping a `>` to `>=`, changing a `+` to a `-`, substituting `True` for `False` — and then running the full test suite against each mutant.



A high mutation kill rate (80%+) indicates a test suite that genuinely detects faults. A low kill rate reveals tests that execute code without meaningfully asserting on its behaviour — coverage without comprehension.

```
# pip install mutmut
# mutmut run --paths-to-mutate src/
# mutmut results

# Example output showing a surviving mutant:
#
# --- src/preprocessing.py
# +++ src/preprocessing.py (mutation id: 42)
# @@ -15,7 +15,7 @@
#  def normalise(score: float) -> float:
#  -     return (score - min_val) / (max_val - min_val)
#  +     return (score - min_val) * (max_val - min_val)    # / changed to *
#
# SURVIVED — no test detected this fault.
# Action: write a test that verifies the output value, not just the type.
```

Chapter 9: Dealing with Hard-to-Test Code

9.1 Why Some Code Is Difficult to Test

Certain code structures are inherently resistant to unit testing. Code that has direct dependencies on external systems (file system, network, clock, randomness), code that mixes business logic with I/O, and code with deeply entangled responsibilities all present testing challenges. The standard refactoring strategies — Peel, Slice, and Monkey Patching — address these challenges by restructuring code to separate testable logic from untestable infrastructure.

9.2 The Peel Strategy

The Peel strategy (also called Extract Method) moves the testable business logic out of a function that contains difficult dependencies, into a separate, pure helper function that can be tested in isolation. The outer function retains the difficult dependency but becomes so thin that it requires no testing itself.

```
# — BEFORE Peel: logic and I/O are entangled —————
def process_and_save(filepath: str) -> None:
    with open(filepath) as f:          # hard dependency: file system
        data = f.read()
        lines = data.strip().split('\n') # business logic (testable)
        filtered = [l for l in lines if l and not l.startswith('#')]
        result = '\n'.join(filtered)
        with open(filepath + '.out', 'w') as f: # hard dependency
            f.write(result)

# Difficult to test without real files. Cannot test logic in isolation.

# — AFTER Peel: logic extracted into a pure, testable function —————
```

```
def filter_comments(lines: list[str]) -> list[str]:
    """Pure function: no I/O, no side effects. Fully testable."""
    return [line for line in lines if line and not line.startswith('#')]

def process_and_save(filepath: str) -> None:
    with open(filepath) as f:
        data = f.read()
        result = '\n'.join(filter_comments(data.strip().split('\n'))) #
    delegate
    with open(filepath + '.out', 'w') as f:
        f.write(result)

# Now test only the extracted logic – no files required:
def test_filter_comments_removes_comment_lines():
    lines = ['# header', 'data_line_1', '', '# comment', 'data_line_2']
    result = filter_comments(lines)
    assert result == ['data_line_1', 'data_line_2']
```

9.3 The Slice Strategy

The Slice strategy (also called Extract Method with Side Effects) extracts a section of code that produces side effects into its own function, creating a clean boundary between logic and I/O. Where Peel extracts the logic out of an I/O function, Slice extracts the I/O out of a logic function:

```
# — BEFORE Slice: timestamps and I/O pollute an otherwise simple
function
def record_model_result(model_name: str, accuracy: float) -> None:
    import datetime
    timestamp = datetime.datetime.now().isoformat() # non-deterministic
    entry = f'{timestamp} {model_name} {accuracy:.4f}\n'
    with open('results.log', 'a') as f: # I/O side effect
        f.write(entry)

# Difficult to test: output depends on current time; writes to real file.

# — AFTER Slice: format logic is extracted and made testable ———
def format_result_entry(model_name: str, accuracy: float,
                        timestamp: str) -> str:
    """Pure formatting function: deterministic, no side effects."""
    return f'{timestamp} {model_name} {accuracy:.4f}\n'

def record_model_result(model_name: str, accuracy: float) -> None:
    import datetime
    timestamp = datetime.datetime.now().isoformat()
    entry = format_result_entry(model_name, accuracy, timestamp) #
    testable
    with open('results.log', 'a') as f:
        f.write(entry)

# Test the extracted logic with a fixed timestamp:
def test_format_result_entry_produces_correct_format():
    entry = format_result_entry('RandomForest', 0.9231, '2024-11-15T10:00:00')
    assert entry == '2024-11-15T10:00:00 RandomForest 0.9231\n'
```

9.4 Monkey Patching

Monkey patching is the runtime substitution of a module attribute, class method, or function with a different implementation. In testing, it replaces a hard-to-control dependency (`datetime.now()`, `random.random()`, `requests.get()`) with a controlled substitute for the duration of a test. Python's `unittest.mock.patch` provides this capability cleanly:

```
from unittest.mock import patch
import datetime

# — Patch datetime.datetime.now to return a fixed timestamp —————
def test_log_entry_contains_correct_timestamp():
    fixed_time = datetime.datetime(2024, 11, 15, 10, 0, 0)

    with patch('mymodule.datetime') as mock_dt:
        mock_dt.datetime.now.return_value = fixed_time
        entry = record_model_result('XGBoost', 0.9412)

        assert '2024-11-15T10:00:00' in entry

# — Patch requests.get to avoid real HTTP calls —————
def test_fetch_model_metrics_parses_response_correctly():
    mock_response = MagicMock()
    mock_response.json.return_value = {'accuracy': 0.95, 'f1': 0.94}
    mock_response.status_code = 200

    with patch('requests.get', return_value=mock_response):
        metrics = fetch_model_metrics('http://mlflow.internal/runs/123')

    assert metrics['accuracy'] == 0.95
```

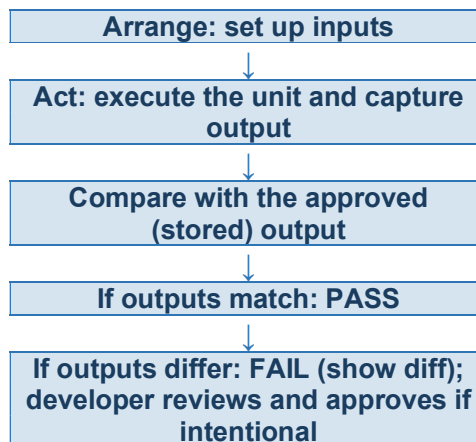
Caution: Monkey patching creates fragile tests that break when the internal structure of the code being patched changes. It should be used sparingly and only when refactoring (Peel/Slice) is not practical. Over-reliance on patching often signals that the production code has too many hard-coded dependencies and would benefit from dependency injection.

Chapter 10: Advanced Testing Techniques

10.1 Approval Testing

Approval testing (also known as snapshot testing or golden master testing) is a technique in which the output of a function is compared against a pre-approved, stored reference output rather than being computed from explicit assertions. It is particularly useful when the output is complex, large, or has many attributes that would require dozens of individual assertions to verify exhaustively.

The approval testing workflow is:



Approval testing is well-suited to: report generation functions, HTML/JSON/XML rendering, complex data transformation outputs, and API response bodies. The `approvaltests` library provides Python support:

```
# pip install approvaltests
from approvaltests import verify

def generate_model_report(model_name, metrics):
    return (
        f'Model: {model_name}\n'
        f'Accuracy: {metrics["accuracy"]:.4f}\n'
        f'Precision: {metrics["precision"]:.4f}\n'
        f'Recall: {metrics["recall"]:.4f}\n'
        f'F1: {metrics["f1"]:.4f}\n'
    )

def test_model_report_format():
    metrics =
    {'accuracy':0.9412,'precision':0.938,'recall':0.944,'f1':0.941}
    report = generate_model_report('XGBoost', metrics)

    # On first run: creates 'approved' file for review
    # On subsequent runs: compares output to the approved file
    verify(report)  # PASS if identical to approved; FAIL with diff if
different
```

10.2 Self-Initialising Fakes

A self-initialising fake (also called a recorded fake or seam fake) is a test double that records real interactions with an external service on its first run, then replays those recorded responses on all subsequent runs. This technique combines the authenticity of real interactions with the speed and determinism of fakes.

Approach	Speed	Authenticity	Requires Network	Maintenance
Real dependency	Slow	Highest	Yes	Low
Manual stub	Fast	Low (hand-written)	No	High (must update manually)
Self-initialising fake	Fast	High (recorded from real)	No (after recording)	Medium (re-record when API changes)

Libraries such as VCR.py (for HTTP interactions) and responses (for requests) implement this pattern for network calls. The recorded responses are stored as YAML or JSON fixture files in the repository:

```
# pip install vcrpy
import vcr

@vcr.use_cassette('fixtures/vcr_cassettes/model_api_response.yaml')
def test_model_api_returns_correct_prediction():
    # First run: makes real HTTP call and records response to the
    # cassette file
    # Subsequent runs: replays the recorded response — no network call
    response = requests.get('https://api.mlplatform.com/predict',
                            json={'features': [5.1, 3.5, 1.4, 0.2]})
    result = response.json()
    assert result['label'] == 'setosa'
    assert 0.0 <= result['confidence'] <= 1.0
```

10.3 Testing Summary — Choosing the Right Tool

Effective testing requires matching the technique to the situation. The following decision guide summarises when each approach is most appropriate:

Situation	Recommended Approach
Testing a pure function (no I/O, no side effects)	Parameterised pytest tests with multiple input/output pairs
Testing a function that calls an external API	Mock or stub the HTTP client; use VCR.py for self-initialising fakes
Testing a function that writes to a database	Use a Fake (in-memory database) or a real test database with fixture teardown
Testing that a function calls its collaborator correctly	Mock with <code>assert_called_once_with()</code>
Testing complex report or formatted output	Approval testing with <code>approvaltests.verify()</code>

Testing a function that is hard to test due to dependencies	Peel or Slice to extract testable logic; use Monkey Patching as last resort
Verifying test quality beyond coverage	Mutation testing with mutmut
Ensuring nothing was broken by a recent change	Regression testing via CI on every commit

SECTION 2 — PRACTICAL & CODING

Chapter 11: Hands-On Notebook — Testing Python Code

The following cells constitute a complete, executable Jupyter notebook that progressively builds a tested Python codebase from scratch. Each cell is explained comprehensively, covering what the code does, why it is structured as it is, and what to observe in the output.

Cell 1 — Setup: Installing Testing Tools

```
# Cell 1: Install the testing toolchain
# Run once per environment

!pip install pytest pytest-cov mutmut approvaltests --quiet

# Verify installations
import subprocess

tools = ['pytest', 'coverage', 'mutmut']
for tool in tools:
    result = subprocess.run([tool, '--version'], capture_output=True,
text=True)
    print(f'{tool}: {result.stdout.strip() or result.stderr.strip()}')
```

Explanation

This cell installs the complete testing toolchain for this notebook:

- **pytest:** The primary test runner and framework. Discovers and executes test functions named `test_*`.
- **pytest-cov:** The pytest plugin that integrates coverage.py, enabling `--cov` flags during test runs to measure statement and branch coverage.
- **mutmut:** A mutation testing tool for Python. Automatically generates code mutations and runs the test suite to measure the mutation kill rate.
- **approvaltests:** The approval testing library. Provides `verify()` and related functions for snapshot-based testing of complex outputs.

The `subprocess.run()` calls verify that each tool is correctly installed and accessible from the system PATH — a useful sanity check that catches common installation issues (wrong Python environment, missing PATH entries) before they cause confusing errors later.

Cell 2 — The Code Under Test: A Data Preprocessing Module

```
# Cell 2: Write the production code we will test
# Save as preprocessing.py for use in subsequent cells

preprocessing_code = '''
"""
Data preprocessing utilities for ML pipelines.

This module provides functions for cleaning, validating, and transforming
tabular data prior to model training or inference.
"""
from __future__ import annotations
import statistics
from typing import Any

class ValidationError(Exception):
    """Raised when input data fails validation."""

def validate_scores(scores: list[Any]) -> list[float]:
    """
    Validate and convert a list of raw score values to floats.

    Parameters
    -----
    scores : list[Any]
        Raw score values. Must be non-empty; each must be numeric.

    Returns
    -----
    list[float]
        Validated scores as floats.

    Raises
    -----
    ValidationError
        If scores is empty or contains non-numeric values.
    """
    if not scores:
        raise ValidationError('\ Scores list must not be empty.\ ')
    result = []
    for i, s in enumerate(scores):
        try:
            result.append(float(s))
        except (TypeError, ValueError):
            raise ValidationError(f'\ Score at index {i} is not numeric:
{s!r}\ ')
    return result

def normalise_min_max(scores: list[float]) -> list[float]:
    """
    Normalise scores to the [0, 1] range using min-max scaling.
    """

```

```
Parameters
-----
scores : list[float]
    A list of at least two distinct values.

Returns
-----
list[float]
    Min-max normalised values in [0, 1].

Raises
-----
ValidationError
    If all scores are identical (division by zero would occur).
"""
min_val = min(scores)
max_val = max(scores)
if max_val == min_val:
    raise ValidationError('\Cannot normalise: all scores are
identical.\')
return [(s - min_val) / (max_val - min_val) for s in scores]

def classify_scores(scores: list[float], threshold: float = 70.0) ->
list[str]:
    """
    Classify each score as PASS or FAIL relative to a threshold.

    Parameters
    -----
    scores : list[float]
        The scores to classify.
    threshold : float
        The minimum score for a PASS classification. Default 70.0.

    Returns
    -----
    list[str]
        A list of \'PASS\' or \'FAIL\' strings.
    """
    return ['PASS\' if s >= threshold else \'FAIL\' for s in scores]

def compute_summary(scores: list[float]) -> dict[str, float]:
    """
    Compute descriptive statistics for a list of scores.

    Returns
    -----
    dict[str, float]
        Keys: mean, median, stdev, min, max.
    """
    return {
        \'mean\' : round(statistics.mean(scores), 4),
        \'median\' : round(statistics.median(scores), 4),
```

```
\ 'stdev\ ' : round(statistics.stdev(scores), 4) if len(scores) > 1
else 0.0,
\ 'min\ ' : min(scores),
\ 'max\ ' : max(scores),
    }
'''

with open('preprocessing.py', 'w') as f:
    f.write(preprocessing_code.replace("\\\ ' " , "'"))

print('preprocessing.py written successfully.')

# Quick smoke test: import and call the module
import importlib.util, sys
spec = importlib.util.spec_from_file_location('preprocessing',
    './preprocessing.py')
pp = importlib.util.module_from_spec(spec)
spec.loader.exec_module(pp)

print('validate_scores([85, 92, 78]):', pp.validate_scores([85, 92, 78]))
print('normalise_min_max([0,50,100]):', pp.normalise_min_max([0.0, 50.0,
100.0]))
print('classify_scores([55,70,85]):', pp.classify_scores([55.0, 70.0,
85.0]))
```

Explanation

This cell defines the production code module that the subsequent test cells will exercise. Its structure embodies several best practices worth noting explicitly:

- **Custom exception class (ValidationError):** Defining a module-specific exception (rather than raising generic ValueError or TypeError) gives callers precise control over error handling: they can catch ValidationError specifically without accidentally masking other exceptions. It also makes error messages in tests far more descriptive.
- **PEP 257 NumPy-format docstrings:** Every function has a complete docstring specifying parameters (name, type, description), return type, and exceptions raised. This docstring serves as the specification against which tests are written — a TDD principle in documentation form.
- **Type annotations on all signatures:** list[Any], list[float], dict[str, float] — all parameters and return types are annotated. This enables mypy to catch type errors before tests run, and helps IDEs provide correct autocomplete.
- **Pure functions:** All four functions are pure: they take inputs and return outputs with no side effects, no global state, no I/O. Pure functions are trivially easy to test — they require no fixtures, no mocks, and no teardown.

Cell 3 — Writing unittest Tests

```
# Cell 3: unittest test suite for preprocessing.py
import unittest
import sys
sys.path.insert(0, '.')
```

```
from preprocessing import validate_scores, normalise_min_max,
classify_scores
from preprocessing import compute_summary, ValidationError

class TestValidateScores(unittest.TestCase):
    """Tests for validate_scores()."""

    # — Happy path —————
    def test_valid_int_list_converts_to_float_list(self):
        result = validate_scores([85, 92, 78])
        self.assertEqual(result, [85.0, 92.0, 78.0])

    def test_valid_string_numbers_are_converted(self):
        result = validate_scores(['85.5', '92', '78.0'])
        self.assertAlmostEqual(result[0], 85.5, places=5)

    def test_single_element_list_is_valid(self):
        result = validate_scores([42])
        self.assertEqual(result, [42.0])

    # — Error cases —————
    def test_empty_list_raises_validation_error(self):
        with self.assertRaises(ValidationError):
            validate_scores([])

    def test_non_numeric_value_raises_validation_error(self):
        with self.assertRaises(ValidationError):
            validate_scores([85, 'N/A', 78])

    def test_none_value_raises_validation_error(self):
        with self.assertRaises(ValidationError):
            validate_scores([85, None, 78])

class TestNormaliseMinMax(unittest.TestCase):
    """Tests for normalise_min_max()."""

    def test_min_value_normalises_to_zero(self):
        result = normalise_min_max([0.0, 50.0, 100.0])
        self.assertAlmostEqual(result[0], 0.0, places=9)

    def test_max_value_normalises_to_one(self):
        result = normalise_min_max([0.0, 50.0, 100.0])
        self.assertAlmostEqual(result[2], 1.0, places=9)

    def test_midpoint_normalises_to_half(self):
        result = normalise_min_max([0.0, 50.0, 100.0])
        self.assertAlmostEqual(result[1], 0.5, places=9)

    def test_all_identical_values_raises_validation_error(self):
        with self.assertRaises(ValidationError):
            normalise_min_max([5.0, 5.0, 5.0])
```

```
class TestClassifyScores(unittest.TestCase):
    """Tests for classify_scores()."""

    def test_score_above_threshold_is_pass(self):
        self.assertEqual(classify_scores([85.0]), ['PASS'])

    def test_score_exactly_at_threshold_is_pass(self):
        self.assertEqual(classify_scores([70.0]), ['PASS'])

    def test_score_below_threshold_is_fail(self):
        self.assertEqual(classify_scores([69.9]), ['FAIL'])

    def test_custom_threshold_applied_correctly(self):
        result = classify_scores([55.0, 90.0], threshold=60.0)
        self.assertEqual(result, ['FAIL', 'PASS'])

# Run the suite directly
loader = unittest.TestLoader()
suite = unittest.TestSuite()
for cls in [TestValidateScores, TestNormaliseMinMax, TestClassifyScores]:
    suite.addTests(loader.loadTestsFromTestCase(cls))

runner = unittest.TextTestRunner(verbosity=2)
result = runner.run(suite)
```

Explanation

This cell demonstrates the complete unittest style applied to the preprocessing module. Several design decisions are worth highlighting:

- **Test class per function:** `TestValidateScores`, `TestNormaliseMinMax`, `TestClassifyScores` each focus on a single function. This organisation makes it easy to find all tests for a given function, and ensures that `setUp/tearDown` logic (when needed) is specific to one unit.
- **Happy path and error case separation:** Each test class contains separate tests for successful inputs ('happy path') and invalid/edge-case inputs ('error cases'). This structure ensures both branches of the code's validation logic are exercised — a branch coverage discipline.
- **`assertAlmostEqual()` for floats:** Floating-point arithmetic in min-max normalisation produces values like 0.49999999999 rather than exactly 0.5. `assertAlmostEqual(result, 0.5, places=9)` tolerates rounding differences up to 9 decimal places, making the test robust without being imprecise.
- **Programmatic test suite assembly:** The `unittest.TestLoader` and `TestSuite` are used to programmatically collect and run tests within the notebook — normally `pytest` handles this automatically from the command line. This demonstrates how test infrastructure works under the hood.

Cell 4 — Writing `pytest` Tests with Fixtures & Parameterisation

```
# Cell 4: pytest test suite – same module, richer test patterns
```

```
# Save as test_preprocessing_pytest.py

pytest_code = '''
"""
pytest test suite for the preprocessing module.
Demonstrates fixtures, parametrize, and exception testing.
"""
import pytest
from preprocessing import (
    validate_scores, normalise_min_max, classify_scores,
    compute_summary, ValidationError
)

# — Fixtures —————
@pytest.fixture
def sample_scores():
    """Standard exam score dataset for reuse across tests."""
    return [55.0, 70.0, 78.5, 85.0, 92.0, 100.0]

@pytest.fixture
def validated_scores(sample_scores):
    """Pre-validated version of sample_scores (fixture composition)."""
    return validate_scores(sample_scores)

# — Parameterised validate_scores tests —————
@pytest.mark.parametrize(
    \ 'raw_input, expected\ ',
    [
        ([1, 2, 3], [1.0, 2.0, 3.0]), # integers
        ([\ '85\ ', \ '92\ '], [85.0, 92.0]), # numeric strings
        ([0], [0.0]), # single element
        ([100.0, 0.0], [100.0, 0.0]), # floats
    ]
)
def test_validate_scores_converts_valid_input(raw_input, expected):
    assert validate_scores(raw_input) == expected

@pytest.mark.parametrize(
    \ 'invalid_input\ ',
    [
        ([]), # empty list
        ([85, \ 'N/A\ ', 78]), # non-numeric string
        ([85, None, 78]), # None value
        ([\ 'not_a_number\ ']), # single bad value
    ]
)
def test_validate_scores_raises_for_invalid_input(invalid_input):
    with pytest.raises(ValidationError):
        validate_scores(invalid_input)

# — Fixture-based normalisation tests —————
def test_normalise_min_is_zero(validated_scores):
```

```
    result = normalise_min_max(validated_scores)
    assert min(result) == pytest.approx(0.0, abs=1e-9)

def test_normalise_max_is_one(validated_scores):
    result = normalise_min_max(validated_scores)
    assert max(result) == pytest.approx(1.0, abs=1e-9)

def test_normalise_all_values_in_unit_interval(validated_scores):
    result = normalise_min_max(validated_scores)
    assert all(0.0 <= v <= 1.0 for v in result)

# — Parameterised boundary tests for classify_scores —————
@pytest.mark.parametrize(
    'score, threshold, expected',
    [
        (70.0, 70.0, 'PASS'), # exactly at threshold → PASS
        (69.9, 70.0, 'FAIL'), # just below → FAIL
        (100.0, 70.0, 'PASS'), # maximum → PASS
        (0.0, 70.0, 'FAIL'), # minimum → FAIL
        (60.0, 60.0, 'PASS'), # custom threshold → PASS
    ]
)
def test_classify_score_boundary_cases(score, threshold, expected):
    result = classify_scores([score], threshold=threshold)
    assert result[0] == expected

# — Summary statistics tests —————
def test_compute_summary_mean_is_correct(sample_scores):
    summary = compute_summary(sample_scores)
    expected_mean = sum(sample_scores) / len(sample_scores)
    assert summary['mean'] == pytest.approx(expected_mean, rel=1e-4)

def test_compute_summary_contains_all_keys(sample_scores):
    summary = compute_summary(sample_scores)
    assert set(summary.keys()) == {'mean', 'median', 'stdev',
    'min', 'max'}
'''

with open('test_preprocessing_pytest.py', 'w') as f:
    f.write(pytest_code.replace("\\\\'", "'"))

print('test_preprocessing_pytest.py written.')

# Run pytest programmatically and capture output
result = subprocess.run(
    ['python', '-m', 'pytest', 'test_preprocessing_pytest.py', '-v', '--
tb=short'],
    capture_output=True, text=True
)
print(result.stdout)
if result.returncode != 0:
    print('STDERR:', result.stderr)
```

Explanation

This cell re-tests the same module using pytest patterns, revealing the advantage of pytest's fixture and parametrize system over unittest's class-based approach:

- **Fixture composition (validated_scores depends on sample_scores):** The validated_scores fixture calls validate_scores(sample_scores) — consuming the sample_scores fixture. pytest resolves this dependency automatically. This composability means fixture logic can be layered without any boilerplate: a test that needs pre-validated data just declares validated_scores as a parameter.
- **@pytest.mark.parametrize with multiple invalid inputs:** The test_validate_scores_raises_for_invalid_input parametrize block runs the same exception-verification logic against four different invalid inputs — empty list, non-numeric string, None value, and bad single value. This is the correct way to achieve comprehensive boundary coverage without four separate test functions.
- **pytest.approx() for floating-point comparisons:** pytest.approx(0.0, abs=1e-9) is pytest's float comparison utility. It passes if the actual value is within 1e-9 (absolute tolerance) of 0.0. This is cleaner than assertAlmostEqual and more explicit about the tolerance being applied.
- **pytest.approx(expected_mean, rel=1e-4) with relative tolerance:** rel=1e-4 specifies a relative tolerance — the actual value must be within 0.01% of the expected value. Relative tolerance is more appropriate than absolute tolerance when the magnitude of the value varies across test cases.

Cell 5 — Test Doubles in Action

```
# Cell 5: Demonstrate test doubles using unittest.mock
from unittest.mock import MagicMock, patch, call
import unittest

# — The production code to test —————
class DataPipeline:
    """
    An ML data pipeline that loads data, preprocesses it,
    and notifies a reporting service with the results.
    """

    def __init__(self, data_loader, preprocessor, reporter):
        self._loader = data_loader
        self._preprocessor = preprocessor
        self._reporter = reporter

    def run(self, source: str) -> dict:
        """Execute the full pipeline and return a summary."""
        raw_data = self._loader.load(source)          # I/O: load
        data     = self._preprocessor.process(raw_data) # transform
        processed = self._preprocessor.process(raw_data)
        summary   = processed['summary']
        self._reporter.send_report(summary)           # I/O:
        notify
```

```
        return summary

# — Test using Stubs and Mocks —————
class TestDataPipeline(unittest.TestCase):

    def setUp(self):
        # Stub: data_loader always returns the same controlled data
        self.stub_loader = MagicMock()
        self.stub_loader.load.return_value = {
            'records': [{'score': 85}, {'score': 92}, {'score': 78}]
        }

        # Stub: preprocessor returns a controlled summary
        self.stub_preprocessor = MagicMock()
        self.stub_preprocessor.process.return_value = {
            'summary': {'mean': 85.0, 'max': 92.0, 'min': 78.0}
        }

        # Mock: reporter — we will verify it was called correctly
        self.mock_reporter = MagicMock()

        self.pipeline = DataPipeline(
            data_loader=self.stub_loader,
            preprocessor=self.stub_preprocessor,
            reporter=self.mock_reporter,
        )

    def test_pipeline_returns_correct_summary(self):
        result = self.pipeline.run(source='data/train.csv')
        self.assertEqual(result['mean'], 85.0)
        self.assertEqual(result['max'], 92.0)

    def test_pipeline_calls_loader_with_correct_source(self):
        self.pipeline.run(source='data/train.csv')
        # Verify stub was called with the expected argument
        self.stub_loader.load.assert_called_once_with('data/train.csv')

    def test_pipeline_sends_report_exactly_once(self):
        self.pipeline.run(source='data/train.csv')
        # Mock assertion: verify the reporter was called exactly once
        self.mock_reporter.send_report.assert_called_once()

    def test_pipeline_sends_correct_summary_in_report(self):
        self.pipeline.run(source='data/train.csv')
        # Mock assertion: verify the report content
        expected_summary = {'mean': 85.0, 'max': 92.0, 'min': 78.0}
        self.mock_reporter.send_report.assert_called_once_with(expected_summary)

# Run
runner = unittest.TextTestRunner(verbosity=2)
suite = unittest.TestLoader().loadTestsFromTestCase(TestDataPipeline)
runner.run(suite)
```

Explanation

This cell is the central demonstration of test doubles — specifically how stubs and mocks work together to test a unit that has multiple external dependencies. The `DataPipeline` class has three collaborators: a data loader (network/database I/O), a preprocessor (CPU computation), and a reporter (network/notification I/O). None of these should be real in a unit test.

- **MagicMock() as a stub (stub_loader, stub_preprocessor):** `MagicMock` auto-generates all attributes and methods. Setting `.return_value` on a method configures what it returns when called. The stubs give the pipeline controlled, deterministic inputs — eliminating I/O and non-determinism.
- **MagicMock() as a mock (mock_reporter):** The same `MagicMock` is now used as a mock — but instead of configuring return values, we use its call-recording capability to assert what the pipeline did to the reporter. `assert_called_once_with(expected_summary)` verifies that `send_report` was called exactly once and with precisely the expected argument. Any deviation — wrong argument, called twice, never called — fails the test.
- **setUp() as fixture for mock wiring:** All mock configuration happens in `setUp()`, creating a clean, identical set of doubles before each test. This ensures tests are independent: a mock assertion in one test does not bleed into the next.
- **Four separate tests for four distinct behaviours:** Return value, loader call signature, reporter call count, reporter call content — each is a separate test case. If any one of these breaks, the failing test immediately identifies which aspect of the pipeline's behaviour regressed.

Cell 6 — Code Coverage Report

```
# Cell 6: Measure test coverage of preprocessing.py

result = subprocess.run(
    [
        'python', '-m', 'pytest',
        'test_preprocessing_pytest.py',
        '--cov=preprocessing',
        '--cov-branch',          # enable branch coverage
        '--cov-report=term-missing', # show uncovered lines
        '-q',                    # quiet mode (less noise)
    ],
    capture_output=True, text=True
)

print('=== Coverage Report ===')
print(result.stdout)

# Parse the coverage percentage from the output
import re
match = re.search(r'TOTAL\s+\d+\s+\d+\s+(\d+)\%', result.stdout)
if match:
    coverage_pct = int(match.group(1))
    print(f'Overall Coverage: {coverage_pct}%')
    if coverage_pct >= 90:
```

```
        print('Coverage target met.')
    else:
        print(f'Coverage below 90% target. Add tests for uncovered
lines.')
```

Explanation

This cell runs the pytest suite with coverage measurement and produces a detailed, actionable report. Understanding coverage report output is a core data engineering skill:

- **--cov=preprocessing:** Specifies the module to measure. Without this, pytest-cov would attempt to measure coverage of all imported code (including the standard library), producing meaningless percentages.
- **--cov-branch:** Enables branch coverage in addition to statement coverage. The report will show branches that were taken and branches that were never exercised, revealing missing test scenarios for decision points.
- **--cov-report=term-missing:** Adds a 'Missing' column to the terminal report listing the specific line numbers not covered by any test. These line numbers are the exact actionable targets for additional tests.
- **Coverage percentage parsing:** The regex extraction of the coverage percentage enables programmatic decision-making: the script evaluates whether the 90% target has been met and prints a structured result. In CI/CD pipelines, this logic is handled by `--cov-fail-under`, which causes pytest to exit with code 1 (failing the build) if coverage is below the threshold.

Cell 7 — TDD in Practice: Building a Feature Test-First

```
# Cell 7: TDD demonstration — build a z-score normalisation function
# Workflow: Red → Green → Refactor

import statistics

# — STEP 1: RED — Write the test first —————
# The function z_normalise() does not exist yet.
# We are defining its contract through tests before writing any code.

def test_z_normalise_mean_of_output_is_zero():
    result = z_normalise([10.0, 20.0, 30.0])
    mean = sum(result) / len(result)
    assert abs(mean) < 1e-9, f'Expected mean near 0, got {mean}'

def test_z_normalise_stdev_of_output_is_one():
    result = z_normalise([10.0, 20.0, 30.0])
    std = statistics.stdev(result)
    assert abs(std - 1.0) < 1e-9, f'Expected stdev near 1, got {std}'

def test_z_normalise_known_value():
    # For [0, 10, 20]: mean=10, stdev=10, z-scores=[-1, 0, 1]
    result = z_normalise([0.0, 10.0, 20.0])
    assert abs(result[0] - (-1.0)) < 1e-9
    assert abs(result[1] - 0.0) < 1e-9
    assert abs(result[2] - 1.0) < 1e-9
```

```
# Run tests — they should FAIL (function does not exist yet)
for test_fn in [test_z_normalise_mean_of_output_is_zero,
                test_z_normalise_stdev_of_output_is_one,
                test_z_normalise_known_value]:
    try:
        test_fn()
        print(f'UNEXPECTED PASS: {test_fn.__name__}')
    except NameError as e:
        print(f'FAIL (NameError — expected): {test_fn.__name__}')

# — STEP 2: GREEN — Implement the minimal function —————
def z_normalise(scores: list[float]) -> list[float]:
    """Standardise scores to zero mean and unit variance (z-score)."""
    mu = statistics.mean(scores)
    sigma = statistics.stdev(scores)
    if sigma == 0:
        raise ValueError('Cannot z-normalise: all values are identical.')
    return [(s - mu) / sigma for s in scores]

# Run tests again — they should all PASS
all_passed = True
for test_fn in [test_z_normalise_mean_of_output_is_zero,
                test_z_normalise_stdev_of_output_is_one,
                test_z_normalise_known_value]:
    try:
        test_fn()
        print(f'PASS: {test_fn.__name__}')
    except AssertionError as e:
        print(f'FAIL: {test_fn.__name__} — {e}')
        all_passed = False

print(f'\nAll tests passed: {all_passed}')

# — STEP 3: REFACTOR — (implementation is already clean) —————
# Add edge case test and handle it:
def test_z_normalise_identical_values_raises():
    try:
        z_normalise([5.0, 5.0, 5.0])
        print('FAIL: Should have raised ValueError')
    except ValueError:
        print('PASS: test_z_normalise_identical_values_raises')

test_z_normalise_identical_values_raises()
```

Explanation

This cell is the most pedagogically significant in the notebook: it enacts the TDD Red-Green-Refactor cycle in real time, making the process tangible rather than abstract.

- **Step 1 — Red:** Three test functions are defined for `z_normalise()` — a function that does not yet exist. When run, they fail with `NameError: name 'z_normalise' is not defined`. This is the expected, correct failure: it confirms that the tests are not trivially passing, and that the implementation will be meaningfully tested when written. The tests encode three

distinct properties of a z-score transformation: output mean is 0, output standard deviation is 1, and specific known inputs produce specific known z-scores.

- **Step 2 — Green:** The minimal implementation is written: compute the mean and standard deviation, then apply the z-score formula to each element. No more code than needed. Running the same three test functions now produces three PASSes — confirming the implementation satisfies all three encoded contracts.
- **Step 3 — Refactor + Edge Case:** A fourth test is added for the edge case (all identical values, which would produce division by zero). The implementation already handles this via the `if sigma == 0` guard — the test confirms it. In a longer TDD session, this would be a new Red step (write the test, watch it fail, then add the guard).

Chapter 12: Summary — Testing Reference

This chapter has provided a comprehensive treatment of automated testing in Python — from foundational vocabulary through advanced techniques. The following table consolidates the key concepts:

Concept	One-Sentence Summary
Unit Testing	Test one isolated function/method at a time with no external dependencies
AAA Pattern	Arrange setup, Act on the UUT, Assert on the outcome
Test Fixture	<code>setUp()/tearDown()</code> or <code>@pytest.fixture</code> ensures each test starts from a clean state
unittest	Python's built-in class-based testing framework; no external dependencies
pytest	Modern function-based testing framework; richer fixtures, parametrize, plugins
Parameterised Tests	Run one test logic with many input/output combinations via <code>@pytest.mark.parametrize</code>
TDD	Write a failing test first, then implement minimum code to pass, then refactor
CI	Automatically run tests on every commit; fail the build if tests fail
Regression Testing	Re-run the full test suite after changes to confirm nothing was broken
Dummy	Placeholder that satisfies parameter requirements but is never called
Stub	Returns predefined values to control what the UUT receives

Fake	Lightweight working implementation (e.g. in-memory DB instead of real DB)
Spy	Records calls for later assertion; does not fail proactively
Mock	Specifies and verifies expected interactions; fails immediately on deviation
Code Coverage	% of lines executed during tests; necessary but not sufficient for quality
Branch Coverage	% of decision branches exercised; stronger signal than statement coverage
Mutation Testing	Measures whether tests detect injected bugs; strongest quality metric
Peel / Slice	Refactor patterns to separate testable logic from hard-to-test I/O
Monkey Patching	Runtime dependency substitution via <code>unittest.mock.patch</code>
Approval Testing	Compare complex output against a pre-approved stored snapshot

"Every line of production code should exist because a test required it." — Kent Beck, Test Driven Development: By Example

For production GenAI and data science systems, the testing disciplines covered in this chapter are not optional extras — they are the engineering foundation that enables teams to ship changes confidently, refactor aggressively, and detect regressions automatically before they reach users or corrupt downstream data pipelines.